

브로드캐스팅 vs apply

언제 무엇을 쓰는가 | 광명코딩학원

> 브로드캐스팅은 모든 값에 똑같은 연산 하나를 한 번에 적용하는 방식이고, apply는 값마다 다른 결과가 나올 수 있는 복잡한 함수(조건 분기 등)를 한 개씩 떼어내서 적용하는 방식입니다. 단순 연산/비교는 브로드캐스팅으로 충분하지만, if-elif-else처럼 분기가 필요한 로직은 Series 전체를 한 번에 조건문에 넣을 수 없어서 apply가 필요합니다.

1. 브로드캐스팅 — 단순 연산 (곱하기)

모든 값에 똑같은 연산 규칙 하나만 적용하면 되는 경우, pandas가 내부적으로 한 번에 처리합니다.

```
from pandas import DataFrame

df = DataFrame({
    "이름": ["철수", "영희", "민수"],
    "점수": [85, 92, 78]
})

print(df["점수"] * 2)
# 0    170
# 1    184
# 2    156
# Name: 점수, dtype: int64
```

2. 브로드캐스팅 — 비교 연산

모든 값에 똑같은 조건 비교 하나만 적용합니다. 결과는 True/False로 이루어진 Series입니다.

```
print(df["점수"] >= 90)
# 0    False
# 1     True
# 2    False
# Name: 점수, dtype: bool
```

3. 브로드캐스팅 직접 시도 — if-elif-else (실패하는 경우)

Series 전체를 통째로 if문 조건에 넣으면 에러가 발생합니다. 파이썬 if문은 "숫자 하나"에 대해서만 참/거짓을 판단하는 문법이라서, 여러 개의 값(Series)을 한 번에 판단할 수 없습니다.

```
df["등급"] = "A" if df["점수"] >= 90 else "B"

# ValueError: The truth value of a Series is ambiguous.
# Use a.empty, a.bool(), a.item(), a.any() or a.all().
```

주의: 이 에러가 바로 "조건 분기는 브로드캐스팅으로 단순하게 처리할 수 없다"는 신호입니다.

4. apply — 조건 분기 함수를 값 하나씩 적용

Series의 값을 하나씩 꺼내서 함수에 넣어주고, 그 함수 안에서 일반적인 if-elif-else를 정상적으로 사용합니다.

```
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    else:
        return "C"

df["등급"] = df["점수"].apply(grade)
print(df)
#   이름  점수  등급
# 0  철수   85    B
# 1  영희   92    A
# 2  민수   78    C
```

동작 방식: 85 -> grade(85) -> "B", 92 -> grade(92) -> "A", 78 -> grade(78) -> "C". 한 번에 하나의 숫자만 다루기 때문에 일반 조건문이 정상적으로 동작합니다.

5. (참고) np.select — 브로드캐스팅 방식으로 조건 분기 처리

apply 없이도 조건 분기를 처리하는 방법이 있습니다. 조건 리스트와 결과 리스트를 매칭시켜 한 번에 처리합니다. 코드는 더 복잡하지만 속도는 더 빠릅니다(데이터가 아주 많을 때 유리).

```
import numpy as np

conditions = [df["점수"] >= 90, df["점수"] >= 80]
choices = ["A", "B"]
result = np.select(conditions, choices, default="C")
print(result)
# ['B' 'A' 'C']
```

6. 비교 요약표

구분	의미	가능 여부	예시 코드	결과
브로드캐스팅 (단순 연산)	모든 값에 똑같은 연산 하나를 한 번에 적용	가능	<code>df["점수"] * 2</code>	<code>[170, 184, 156]</code>
브로드캐스팅 (비교 연산)	모든 값에 똑같은 조건 비교 하나를 한 번에 적용	가능	<code>df["점수"] >= 90</code>	<code>[False, True, False]</code>
if-elif-else 직접 시도	Series 전체를 통째로 if문 조건에 넣음	불가능 (ValueError)	<code>if df["점수"] >= 90</code> <code>else ...</code>	에러 발생
apply (조건 분기 함수)	값을 하나씩 꺼내서 함수를 각각 실행	가능	<code>df["점수"]</code> <code>.apply(grade)</code>	<code>[B, A, C]</code>
np.select (참고)	조건 리스트와 결과 리스트를 매칭시켜 처리	가능 (복잡하지만 빠름)	<code>np.select([cond1, cond2], [A, B], default=C)</code>	<code>[B, A, C]</code>

[완료] 브로드캐스팅과 apply의 차이를 학습했습니다. 단순 연산/비교는 브로드캐스팅으로, 조건 분기가 필요한 복잡한 로직은 apply로 처리한다는 기준을 기억하세요.